Everest Auditor — Call QA & Verification Platform

Audits recorded debt-collection calls for quality & FDCPA-sensitive compliance: **recording → transcript → checklist verdicts (PASS/FAIL/NA) + coaching/compliance narrative + objection extraction → human verification**. Built per `PRD-call-qa-platform.md`, no managed orchestrator, ≤ $100/month pilot.

Status

Built in the phased order of PRD §18. Current state:

Phase	Status
0 — Foundations, auth, RBAC (monorepo, Docker, schema, JWT, permission matrix, CRUD, CI)	✅ done + tested
Rate-limit / caps / backoff core (NFR3)	✅ done + tested
2 — Orchestration engine (durable queue, state machine, retry/dead-letter, reconciler)	✅ done + tested
1 — Storage & ingestion (R2 presigned, batch upload, SSE)	✅ done + tested
3 — STT (AssemblyAI client, signed webhook, transcript materialization, reconciler poll)	✅ done + tested
4 — KB & checklist builder (KB upload, default checklist, versioning, rubric distillation)	✅ done + tested
5 — Judge + routing layer + objection clustering (Gemini, escalation/circuit-breaker)	✅ done + tested
6 — Report read + lazy narrative + user notes	✅ done + tested

Phase	Status
7 — Verification service (judgement, recording download, transcript, agreement metric)	✅ done + tested
8 — Eval gate + router tuning + observability metrics	✅ done + tested
Frontend three-pane shell (React+TS+Vite+Tailwind+TanStack Query)	✅ done + builds

The entire backend is complete: 126 tests pass end-to-end against a live Postgres (ruff + mypy --strict clean, 69 modules). Full flow: real audio → diarized transcript → routed verdicts + objections → report + lazy narrative → human verification → eval gate + router tuning. AssemblyAI and Gemini are real clients behind Protocols, with deterministic stubs for local dev / CI when no keys are set — the only deploy-time blanks are the API keys, R2 creds, and the public webhook URL. CI runs lint + types + tests + the eval gate.

The judge is multimodal. `GeminiJudge` (google-genai SDK, `gemini-3.6-flash`, Developer API, extended thinking) receives the **recording audio *and* the transcript**, so it judges tone/empathy/talk-over a transcript can't capture. Anti-bias by design: the judge only *evaluates* (every PASS/FAIL must cite a verbatim quote — no evidence, no fail); the **rewriter is a separate, evidence-gated step** that diverges from the original only where a FAIL has cited evidence, and returns the call unchanged with `already_ideal: true` when there are no FAILs — it never manufactures flaws for a clean call.

Architecture (PRD §6)

Two deployables from one monorepo, both from one portable image:

- **`api`** (`app.main:app`) — REST + RBAC, presigned URL issuer, AssemblyAI webhook receiver, SSE notifier.
- **`worker`** (`app.worker.main`) — durable claim→dispatch loop + reconciler sweep + rate limiter / daily caps.

State lives in **Postgres + pgvector** (jobs/queue, reports, objection vectors) and **Cloudflare R2** (recordings/transcripts 30-day lifecycle; KB/reports retained).
External: **AssemblyAI** (STT, async + webhooks + diarization) and **Gemini** (LLM judge).

The orchestration core is a **Postgres-backed durable queue + explicit state machine** (`PENDING_TRANSCRIPTION → AWAITING_TRANSCRIPT → PENDING_JUDGE → DONE/FAILED`), claimed with `FOR UPDATE SKIP LOCKED` + leases for crash recovery (§8). No ADK/Temporal/etc.

Rate limiting, caps & cost (NFR3 — the operational core)

All limits are **config defaults, env-overridable**, derived from the reference volume.
Full derivation: `docs/RATE_LIMITS_AND_COST.md` .

- **Token-bucket RPM/TPM + concurrency** per provider (`app/ratelimit/buckets.py`) — shapes our traffic *under* vendor quotas so we rarely hit a 429.
- **Per-portfolio daily cap** (`app/ratelimit/caps.py`) — atomic Postgres counter; over-cap work is **deferred, not failed** (zero stranded calls).
- **Exponential backoff + jitter, Retry-After aware** (`app/ratelimit/backoff.py`) — for the 429s that slip through; the durable worker persists the retry ladder across restarts.

Tune any of these via Railway variables (see `.env.example`) — e.g. set `DAILY_CAP_PER_PORTFOLIO=85` for the $100 pilot. No redeploy of logic.

Layout

```
backend/app/
  config.py          # all settings + rate-limit/cap/router defaults (single source)
  db.py              # async SQLAlchemy engine/session
  models/            # §9 schema (RBAC, calls, jobs/queue, reports, objections, ...)
  rbac/              # one permission matrix + is_allowed()  (§2)
  security.py        # JWT mint/verify + current_user dependency
  authz.py           # the single (user, action, resource) authorization gate
  ratelimit/         # buckets + caps + backoff   (NFR3)
  storage.py         # R2 presigned PUT/GET + lifecycle + StorageService (Phase 1/3)
  notifier.py        # Postgres LISTEN/NOTIFY status bridge → SSE (Phase 1)
  orchestration/     # state machine, retry policy, queue, engine, reconciler (Phase 2)
  stt/               # AssemblyAI client + normalized Transcript types (Phase 3)
  checklists/        # default seed, builder/versioning service, rubric distillation (Phase 4)
  api/               # auth, portfolios, agents, uploads, calls, events, webhooks, kb, checklist
  main.py            # api entrypoint
  worker/            # durable loop + reconciler entrypoint
backend/migrations/  # Alembic (async env; 0001 = baseline + pgvector + queue indexes)
backend/tests/       # 72 tests: ratelimit, RBAC, state machine, retry, ingestion, notifier, DB
scripts/setup_r2.py  # one-time: create buckets + apply 30-day lifecycle
docs/RATE_LIMITS_AND_COST.md
Dockerfile.api  Dockerfile.worker  docker-compose.yml  .github/workflows/ci.yml
```

Local development

```
# 1. Bring up Postgres+pgvector (and optionally api/worker)
docker compose up -d db

# 2. Python env
python -m venv .venv && . .venv/Scripts/activate    # Windows; use bin/activate on *nix
pip install -e ".[dev]"

# 3. Migrate + run
cp .env.example .env                                 # fill in secrets as phases need them
alembic upgrade head
uvicorn app.main:app --reload --app-dir backend      # api
python -m app.worker.main                            # worker (separate shell)
```

Frontend

React + TypeScript + Vite + Tailwind + TanStack Query + Zustand, three-pane shell (Portfolios | Agents | Calls/Report) with selection in the URL (§11). Dev-login, batch upload (presign → direct-to-R2 PUT → register), live-ish call status, and the §12 report view (narrative, PASS/FAIL/NA verdicts with evidence timestamps, objections, per-item notes, verification controls, recording download).

```
cd frontend
npm install
npm run dev          # proxies /api → http://localhost:8000
npm run build        # tsc --noEmit + vite build
```

Quality gates (run what CI runs)

```
ruff check backend          # lint
mypy backend/app            # types (strict)
pytest -q                   # unit tests
```

CI (`.github/workflows/ci.yml`) runs all three against a pgvector service container on PR and `main` . Phase 8 adds the **eval gate** (§16.2) as a required check that blocks deploys on judge-quality regressions.